

Prédiction du marché des voitures d'occasion

1 <h2> Analyse de données et apprentissage

2 automatique </h2>

3 <p> Groupe : ELLABANE Hafsa

4 FANNI Mohamed

5 HARRAS H'nia

6 SABRI Nada

7 TALHI Mohammed-Amine </p>



Sommaire

01

Description du projet

02

Description de la base des données

03

Traitement des données

04

Les modèles Machine Learning

01

Description du projet

La prédiction des prix des voitures d'occasion

Objectifs

Analyse de la base de données

Construction d'un modèle prédictif

Analyse des résultats

Description de la base des données

Notre dataset regroupe des données sur les voitures d'occasion, avec les caractéristiques suivantes :

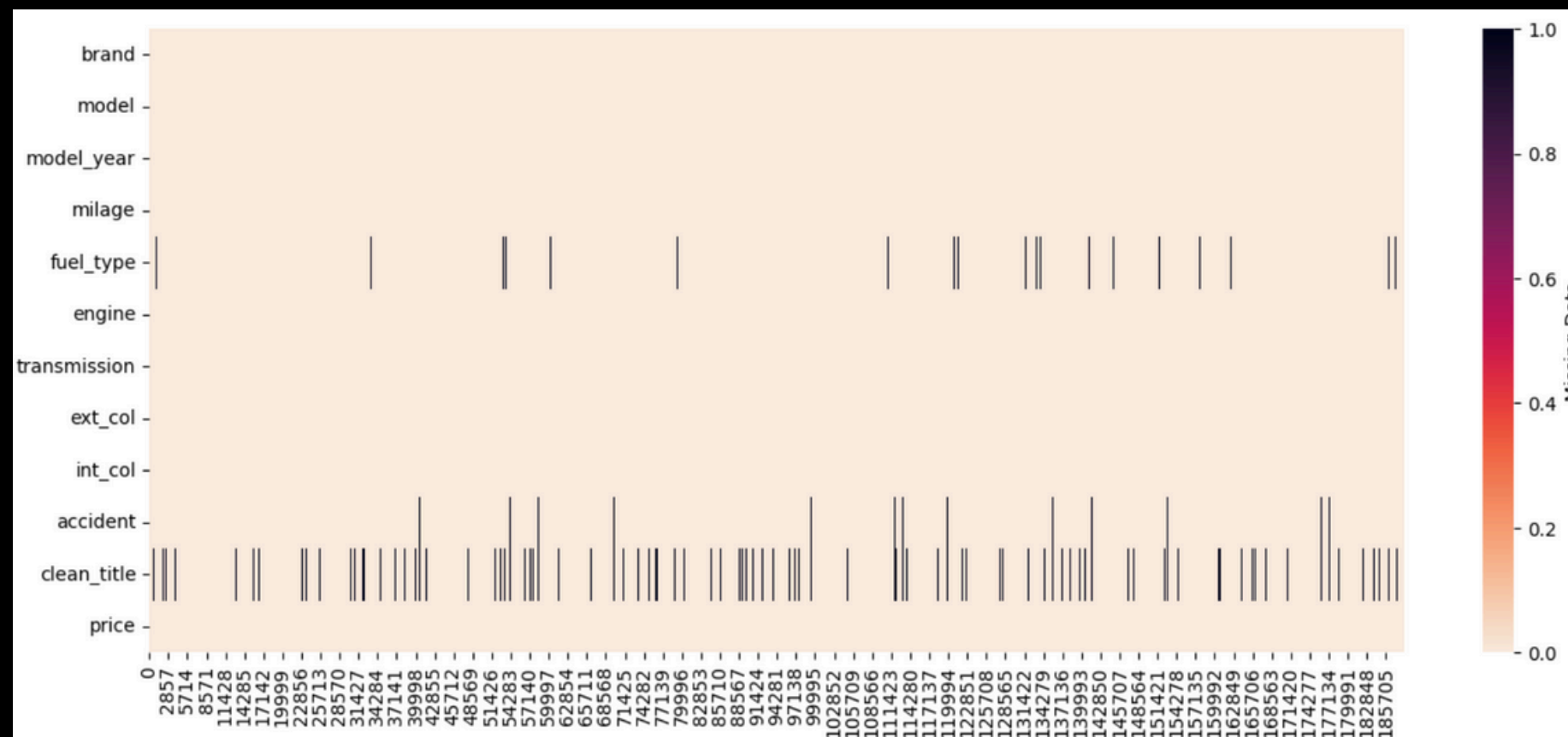
- Marque '**brand**', modèle '**model**', et année '**model_year**' : Identité et ancienneté du véhicule.
- Kilométrage '**milage**' et moteur '**engine**' : Indicateurs d'usure et de performance.
- Type de carburant '**fuel_type**' et transmission '**transmission**' : Spécifications techniques.
- Couleurs (intérieure/extérieure) '**int_color**'/'**ext_color**' : Esthétique du véhicule.
- Historique des accidents '**accident**' et titre '**clean_title**' : État légal et historique.
- Prix '**price**' : Valeur marchande actuelle.

	brand	model	model_year	milage	fuel_type	engine	transmission	ext_col	int_col	accident	clean_title	price
0	MINI	Cooper S Base	2007	213000	Gasoline	172.0HP 1.6L 4 Cylinder Engine Gasoline Fuel	A/T	Yellow	Gray	None reported	Yes	4200
1	Lincoln	LS V8	2002	143250	Gasoline	252.0HP 3.9L 8 Cylinder Engine Gasoline Fuel	A/T	Silver	Beige	At least 1 accident or damage reported	Yes	4999
2	Chevrolet	Silverado 2500 LT	2002	136731	E85 Flex Fuel	320.0HP 5.3L 8 Cylinder Engine Flex Fuel Capab...	A/T	Blue	Gray	None reported	Yes	13900
3	Genesis	G90 5.0 Ultimate	2017	19500	Gasoline	420.0HP 5.0L 8 Cylinder Engine Gasoline Fuel	Transmission w/Dual Shift Mode	Black	Black	None reported	Yes	45000
4	Mercedes-Benz	Metris Base	2021	7388	Gasoline	208.0HP 2.0L 4 Cylinder Engine Gasoline Fuel	7-Speed A/T	Black	Beige	None reported	Yes	97500
...	...	...	...	...	...	...	...	...	...	...	...	...
188528	Cadillac	Escalade ESV Platinum	2017	49000	Gasoline	420.0HP 6.2L 8 Cylinder Engine Gasoline Fuel	Transmission w/Dual Shift Mode	White	Beige	None reported	Yes	27500
188529	Mercedes-Benz	AMG C 43 AMG C 43 4MATIC	2018	28600	Gasoline	385.0HP 3.0L V6 Cylinder Engine Gasoline Fuel	8-Speed A/T	White	Black	At least 1 accident or damage reported	Yes	30000
188530	Mercedes-Benz	AMG GLC 63 Base 4MATIC	2021	13650	Gasoline	469.0HP 4.0L 8 Cylinder Engine Gasoline Fuel	7-Speed A/T	White	Black	None reported	Yes	86900
188531	Audi	S5 3.0T Prestige	2022	13895	Gasoline	3.0L	1-Speed Automatic	Daytona Gray Pearl Effect	Black	None reported	NaN	84900
188532	Porsche	Macan Base	2016	59500	Gasoline	252.0HP 2.0L 4 Cylinder Engine	Transmission w/Dual Shift Mode	White	Black	None reported	Yes	28995

02

Description de la base des données

Heatmap des données manquantes



Colonnes impactées par des données manquantes : **fuel_type**, **accident**, **clean_title**.

La colonne **clean_title** affiche une proportion notable de données manquantes.

CONCLUSION

Bien que certaines colonnes aient peu de valeurs manquantes, la qualité des données globales pourrait nécessiter des ajustements pour garantir un modèle prédictif robuste.

02

Description de la base des données

Cas exceptionnels :

Véhicule avec titre 'yes' dans la colonne `clean_title` malgré un rapport d'accident signalé.

03

Traitement des données

Colonne : Fuel-Type

Gasoline	} Conventional Fuel	<pre>fuel_type_mapping = { 'Gasoline': 1, 'Diesel': 1, 'Hybrid': 2, 'Plug-In Hybrid': 2, 'E85 Flex Fuel': 2, np.nan: 0, '': 0, 'not supported': 0 }</pre>	# Conventional Fuel
Diesel			# Conventional Fuel
Hybrid			# Alternative Fuel/Hybrid
Plug-In Hybrid	} Alternative Fuel/ Hybrid		# Alternative Fuel/Hybrid
E85 Flex Fuel			# Alternative Fuel/Hybrid
NAN	} Unsupported/Invalid/Unknown		# Unsupported/Invalid/Unknown
-			# Unsupported/Invalid/Unknown
Not supported			# Unsupported/Invalid/Unknown

03

Traitement des données

Colonne : Accident

None reported --> 0

At least 1 accident or damage reported --> 1

NAN --> 0.5

```
accident_mapping = {  
    'None reported': 0,  
    'At least 1 accident or damage reported': 1,  
    np.nan : 0.5, # Maybe  
}
```

03 Traitement des données

Colonne : Clean-Title

Yes --> 0

NAN --> 0.5

```
clean_title_mapping = {
    'Yes' : 1,
    np.nan : 0.5,
}
```

03

Traitement des données

Colonne : Transmission

^ A/T
Transmission w/Dual Shift Mode
7-Speed A/T
8-Speed A/T
10-Speed Automatic
1-Speed A/T
6-Speed A/T
10-Speed A/T
9-Speed A/T
8-Speed Automatic

Manuel
M/T

CVT
v

Automatic

Manuel

Continuously variable transmission

```
def map_transmission(trans):  
    trans=trans.lower()  
    if 'automatic' in trans or 'a/t' in trans or 'w/dual shift mode' in trans:  
        return '3'  
    elif 'manual' in trans or 'm/t' in trans:  
        return '2'  
    elif 'cvt' in trans:  
        return '1'  
    else:  
        return '0'
```

Traitement des données

Colonnes : Brand et Model

Les marques de voitures constituent une variable catégorielle avec une grande diversité (de nombreuses valeurs uniques).

Cependant le choix de la nouvelle catégorisation doit conserver l'information économique de la marque de voiture d'où le choix suivant.

Après cette nouvelle catégorie proposée, on estime la non-nécessité de la colonne model

```
tier_mapping = {
    'Rolls-Royce': 5, 'Bentley': 5, 'Ferrari': 5, 'Lamborghini': 5, 'Aston': 5, # Marque Ultra luxueuse
    'McLaren': 5, 'Maybach': 5, 'Bugatti': 5, 'Maserati': 5, 'Porsche': 5,
    'Mercedes-Benz': 4, 'BMW': 4, 'Audi': 4, 'Tesla': 4, 'Genesis': 4, # Marques premium
    'Jaguar': 4, 'Land': 4, 'Lexus': 4, 'Cadillac': 4, 'Alfa': 4,
    'Volvo': 3, 'Acura': 3, 'INFINITI': 3, 'Lincoln': 3, 'Buick': 3, # Marques intermédiaires haut de gamme
    'Polestar': 3, 'Karma': 3, 'Rivian': 3, 'MINI': 3,
    'Toyota': 2, 'Honda': 2, 'Ford': 2, 'Chevrolet': 2, 'Hyundai': 2, # Marques populaires bien établies
    'Volkswagen': 2, 'Subaru': 2, 'Kia': 2, 'GMC': 2, 'Mazda': 2,
    'Nissan': 1, 'Mitsubishi': 1, 'Dodge': 1, 'RAM': 1, 'Chrysler': 1, # Marques économiquement abordables
    'Jeep': 1, 'Suzuki': 1, 'FIAT': 1,
    'Pontiac': 0, 'Saturn': 0, 'Hummer': 0, 'Plymouth': 0, 'Mercury': 0, # Marques discontinuées ou à faible valeur
    'Scion': 0, 'smart': 0, 'Saab': 0, 'Lucid': 0, 'Lotus': 0
}
```

03

Traitement des données

Colonne : Engine

Cette colonne est bien compliqué à interpréter, vu qu'elle présente plusieurs informations sur le modèle.

Exemple: 172.0HP 1.6L 4 Cylinder Engine
Gasoline Fuel

Fuel Type

Horse
Power

Capacité

Type du
cylindre

```
def extract_engine_info(dataset):  
    dataset['horsepower'] = dataset['engine'].str.extract(r'(\d+\.\d+)(?=HP)').astype(float)  
    dataset['engine_capacity'] = dataset['engine'].str.extract(r'(\d+\.\d+)(?=L)').astype(float)  
    dataset['cylinder'] = dataset['engine'].str.extract(r'(\d+)\s(Cylinder|V\d|Straight)')[0].astype(float)  
  
    return dataset.drop(columns=['engine'])
```

03

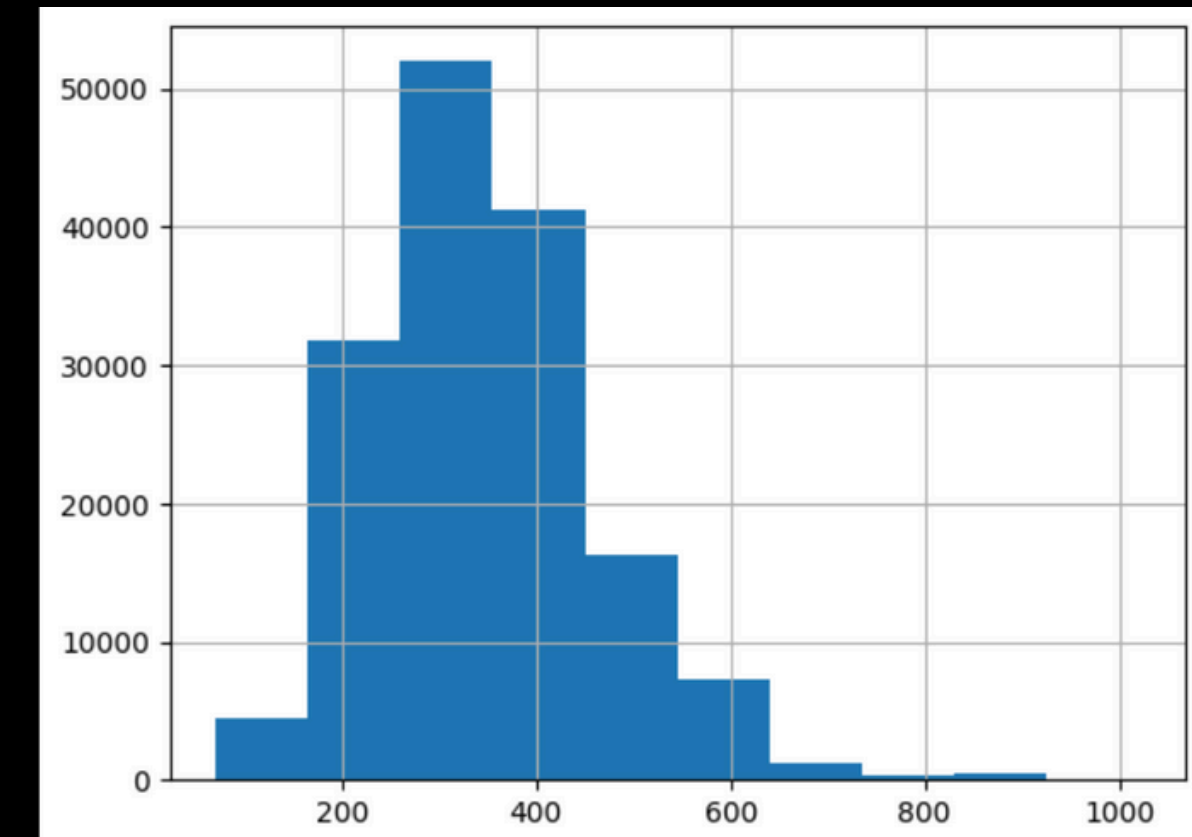
Traitement des données

Colonne : Engine

On représente le histogramme de HorsePower afin de déterminer la valeur avec laquelle on va remplacer les valeurs manquantes.

On remarque que sa distribution est asymétrique, qui reflète que **la moyenne** sera influencé par les valeurs aberrantes (Outliers). En contrepartie, la **médiane** est la valeur centrale des données lorsqu'elles sont triées, et elle n'est pas affectée par les valeurs extrêmes ou les distributions asymétriques.

En suivant la même logique, on remplace **HorsePower** et **cylinder** par **la médiane**, et la valeur **engine_capacity** par **la moyenne**.



HORSEPOWER

```
from sklearn.impute import SimpleImputer

#Replace missing values with the median
sp = SimpleImputer(missing_values=np.nan, strategy='median')
dataset[['horsepower', 'cylinder']] = sp.fit_transform(dataset[['horsepower', 'cylinder']])

#Replace missing values with the mean
sp_ = SimpleImputer(missing_values=np.nan, strategy='mean')
dataset[['engine_capacity']] = sp_.fit_transform(dataset[['engine_capacity']])
```

Traitement des données

Colonnes : ext_col et int_col

Les deux colonnes représentent les couleurs externes et internes. On procède à une dummification de ces deux colonnes. --> **Un grand nombre de colonnes ayant des informations proches.**

Exemple: Gray et Daytona Gray Pearl Effect

On procède donc à utiliser le PCA pour réduire les dimensions des données tout en préservant autant d'information (variance) que possible.

```
from sklearn.decomposition import PCA
```

```
# Step 1: Initialize PCA with the desired number of components
# PCA is set to reduce the data to 4 principal components
pca_ext = PCA(n_components=4)
# Step 2: Fit PCA on the input data and transform it
# The input data `one_hot_encoded_ext_color` is reduced to 4 principal components
pca_data = pca_ext.fit_transform(one_hot_encoded_ext_color)
# Step 3: Create column names for the principal components
# The column names will be PC_ext1, PC_ext2, PC_ext3, and PC_ext4
pca_columns = [f'PC_ext{i+1}' for i in range(4)]
# Step 4: Create a new DataFrame with the transformed PCA data
# The resulting DataFrame has the reduced dimensions as columns
pca_ext_df = pd.DataFrame(data=pca_data, columns=pca_columns)
# Step 5: Get the explained variance ratio for each principal component
# This shows the proportion of variance in the data explained by each component
explained_variance = pca_ext.explained_variance_ratio_
# Step 6: Print the explained variance ratio
# This helps evaluate how much information is retained by each principal component
print(f"Explained variance by each component: {explained_variance}")
```

Explained variance by each component: [0.29103354 0.20367524 0.13048253 0.0979882]

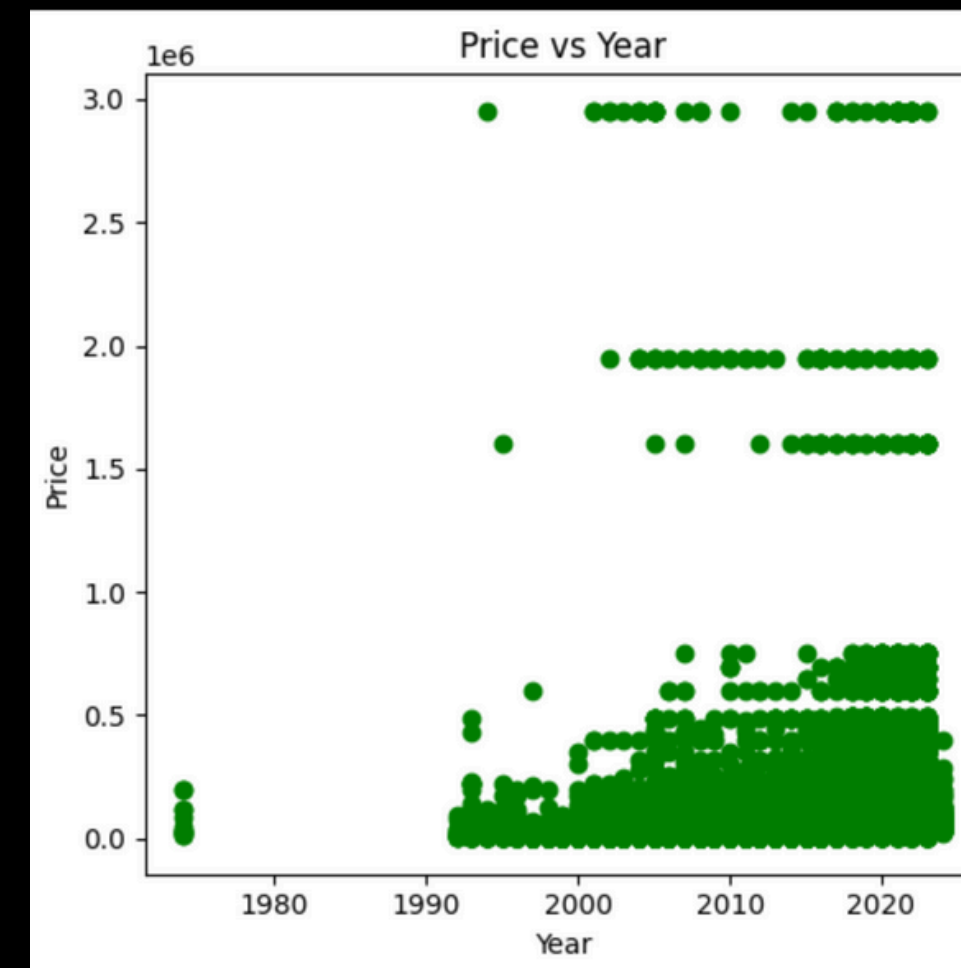
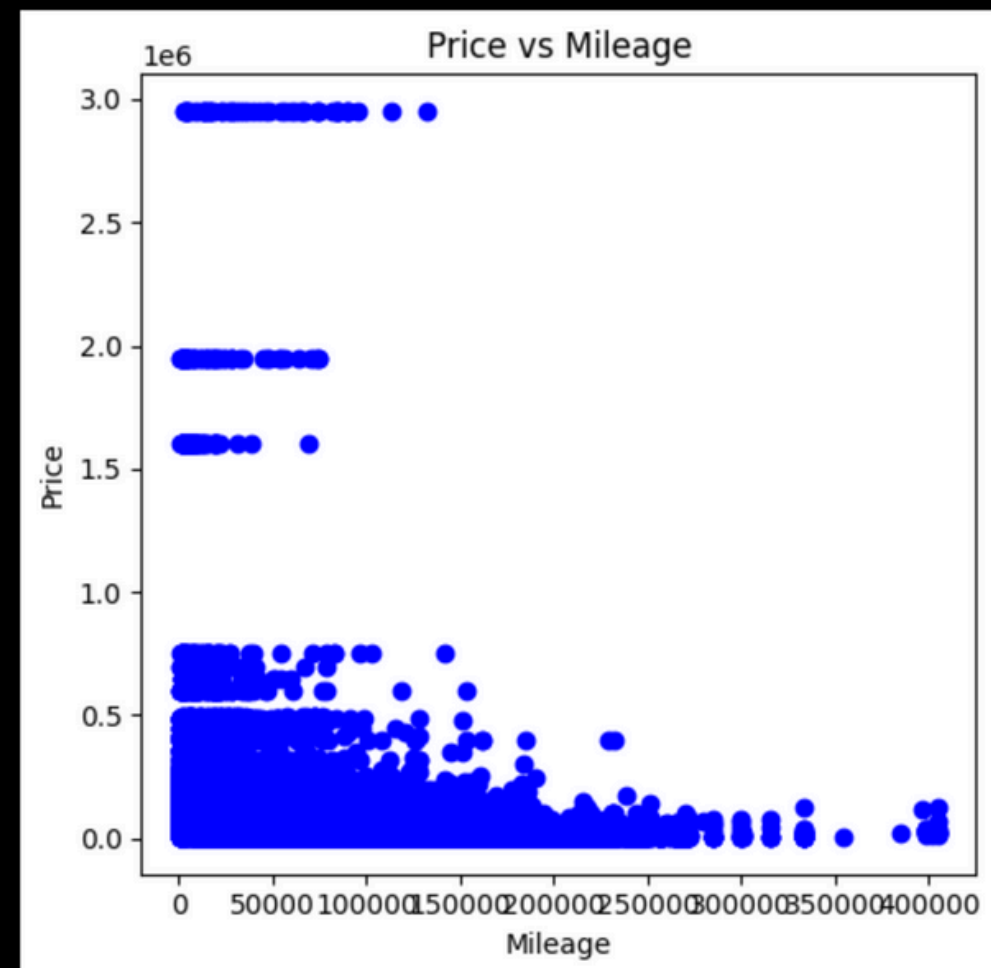
03

Traitement des données

Colonnes : Mileage and year

On procède par la suite à une représentation du prix en fonction du milage et en fonction de l'année.

On remarque que l'élévation du milage entraine une diminution du prix de la voiture. En contrepartie, les modèles récents ont un prix plus élevé.



03

Traitement des données

Colonnes : Mileage and year

Les deux colonnes ajoutées représentent les colonnes numériques de notre data.

On essaie de les mettre à l'échelle en les rendre moins sensibles aux valeurs aberrantes, en utilisant **RobustScaler** de la librairie **sklearn.preprocessing**

```
num_columns = ['milage_per_year', 'age']  
rb = RobustScaler()  
dataset[num_columns] = rb.fit_transform(dataset[num_columns])
```

03

Traitement des données

Colonnes : Mileage and year

Utiliser le kilométrage brut peut être trompeur, car une voiture plus ancienne aura naturellement plus de kilomètres au compteur. --> On propose de la remplacer par la moyenne de milage annuelle.

L'année brute du modèle peut aussi rendre le modèle difficile à interpréter. --> On propose de le remplacer par l'âge de la voiture

```
dataset['milage'] = dataset['milage'] / dataset['age']  
dataset = dataset.rename(columns={'milage': 'milage_per_year'})
```

```
dataset['model_year'] = 2025 - dataset['model_year']  
dataset = dataset.rename(columns={'model_year' : 'age'})
```

03

Traitement des données

Colonnes : Mileage et year

Les deux colonnes ajoutées représentent les colonnes numériques de notre data.

On essaie de les mettre à l'échelle en les rendre moins sensibles aux valeurs aberrantes, en utilisant **RobustScaler** de la librairie **sklearn.preprocessing**

```
num_columns = ['milage_per_year', 'age']  
rb = RobustScaler()  
dataset[num_columns] = rb.fit_transform(dataset[num_columns])
```

04

Les modèles Machine Learning

Choix du modèle

On va essayer avec 3 modèles **GradientBoostingRegressor**, **RandomForestRegressor** et **SVM.SVR**

On va utiliser les 3 trois modèles avec leurs meilleures hyperparamètres et après on va calculer le root-mean-squared-error pour chacun et faire une comparaison entre les 3 modèles

Choix du modèle

↑ On choisit les meilleurs paramètres
du modèle avec le
GradientBoostingRegressor à l'aide
du framework d'optimisation des
hyperparamètres Optuna
↓

```
from sklearn.ensemble import GradientBoostingRegressor
import optuna
import math
from sklearn.model_selection import cross_val_score

#tuning the hyperparameters
def objective(trial):
    params = {
        "n_estimators": trial.suggest_int('n_estimators', 50, 500),
        "learning_rate": trial.suggest_float('learning_rate', 0.01, 0.3),
        "max_depth": trial.suggest_int('max_depth', 2, 10),
        "min_samples_split": trial.suggest_int('min_samples_split', 2, 5),
        "min_samples_leaf": trial.suggest_int('min_samples_leaf', 1, 5),
        "subsample": trial.suggest_float('subsample', 0.4, 0.8)
    }

    model = GradientBoostingRegressor(**params)
    score = cross_val_score(model, X_train, Y_train, n_jobs=-1, cv=3, scoring='neg_mean_squared_error').mean()

    return math.sqrt(-score)

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=100)

best_params = study.best_params
print(best_params)
```

Choix du modèle

^ On choisit les meilleurs paramètres du
| modèle du RandomForestRegressor avec le
| GridSearch
v

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
model = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=3,                    # Number of cross-validation folds
    scoring='neg_mean_squared_error', # Metric to optimize
    verbose=2,               # Display progress during search
    n_jobs=-1                # Use all available processors
)
grid_search.fit(X_train, Y_train)
print("Best Parameters:", grid_search.best_params_)
print("Best Score (MSE):", -grid_search.best_score_)
best_model = grid_search.best_estimator_
```

04

Les modèles Machine Learning

Choix du modèle



Calcul du root-mean-squared-error pour chaque modèle

```
Y_predict_1 = model.predict(X_test)
Y_predict_2 = regressor_.predict(X_test)
Y_predict_3 = best_model.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error
import numpy as np
```

```
rmse_1 = np.sqrt(mean_squared_error(Y_test, Y_predict_1))
rmse_2 = np.sqrt(mean_squared_error(Y_test, Y_predict_2))
rmse_3 = np.sqrt(mean_squared_error(Y_test, Y_predict_3))
```

04

Les modèles Machine Learning

Choix du modèle

↑ L'évaluation des trois modèles par le RMSE
montre que le premier modèle
(`GradientBoostingRegressor` with best
hyperparameters) est le plus performant
↓

```
print(f'RMSE for GradientBoostingRegressor:{rmse_1}')
```

✓ 0.0s

RMSE for GradientBoostingRegressor:62517.92659858001

```
print(f'RMSE for SVR:{rmse_2}')
```

✓ 0.0s

RMSE for SVR:68850.98728129808

```
print(f'RMSE for RandomForestRegressor:{rmse_3}')
```

✓ 0.0s

RMSE for RandomForestRegressor:64105.43913552801

↑
Merci !
↓

1 `<h2> Merci pour votre attention ! </h2>`